MICROKERNEL ARCHITECTURE MANUAL

HYDRA OS

UNIX-like Distributed Microkernel for Modern UEFI Hardware

BOOT SEQUENCE PIPELINE

UEFI POST → GDT/TSS → CPU Diagnostics → Frame Allocator → PML4 Paging → Capability Space → IPC Rendezvous

Edition: v0.1.0 Architecture Specification

Target Architecture: x86_64-unknown-uefi (Native 64-bit Long Mode)

Security Model: Capability-Based Access Control (seL4-style)

IPC Subsystem: Synchronous & Asynchronous Message Rendezvous

Generated: October 22, 2026

Table of Contents — Boot Sequence Topics

SYSTEM OVERVIEW

Architecture Overview & Codebase Index — *Microkernel architecture overview, directory map, and concept analogies.*

BUILD & TOOLCHAIN

Build Automation & Toolchain Configuration — *NASM assembly compiler script, DWARF debug symbols, and UEFI build settings.*

STAGE 0: FIRMWARE HANDOFF

Legacy BIOS Boot Trampoline & Long Mode Switch — *Multiboot2 header, CPUID validation, PAE identity paging, and 32-to-64-bit transition.*

STAGE 1: KERNEL ENTRY POINT

UEFI Entry Point & Bootstrap Orchestrator — *Native UEFI application entry point, system table access, panic handler, and main sequence.*

STAGE 2: EARLY DIAGNOSTICS

UEFI Console & Thread-Safe Output — *Bare-metal formatting macros, spinlock mutex synchronization, and UEFI SimpleTextOutput.*

STAGE 3: CPU ARCHITECTURE

Architecture Abstraction Layer — *Hardware isolation layer separating processor specifics from microkernel core.*

Global Descriptor Table (GDT) & TSS — *Privilege levels (Ring 0 / Ring 3), Task State Segment, and Double Fault IST stack.*

CPU Control Registers & Diagnostics — *Control register inspection (CR0 protection, CR2 fault address, CR3 PML4 root, CR4).*

STAGE 4: MEMORY SUBSYSTEM

Memory Subsystem Coordinator — *Coordinator binding physical frame management and virtual memory translation.*

Physical Memory Discovery & Frame Allocator — *UEFI memory map parser, 1 MiB low memory reservation, and 4 KiB frame allocator.*

STAGE 5: VIRTUAL MEMORY

4-Level Paging & Address Translation — *4-level x86_64 PML4 hierarchy, virtual address translation, and 2 MiB / 1 GiB huge pages.*

STAGE 6: SECURITY & CAPABILITIES

Capability Security Model & Rights — *Object-capability security tokens, Rights bitflags, and CapType kernel resource identifiers.*

Task Capability Space & Table Management	*Per-task CSpace table, 256-slot fixed array storage, rights verification, and revocation.*

STAGE 7: IPC SUBSYSTEM

Inter-Process Communication Overview	*Microkernel message passing coordinating isolated userland services.*
Fast Register Messages & Delegation	*Fast 4-word inline register payload, protocol labels, and transferred capability tokens.*
Rendezvous Channels & Endpoints	*Circular FIFO ring buffers, rendezvous mailboxes, and thread-safe SharedEndpoint.*

SYSTEM OVERVIEW • BOOT SEQUENCE ORDER #01

Hydra OS - Codebase Documentation Index

Welcome to the technical documentation for **Hydra OS**. Every source code file in the repository is documented with real-world analogies, conceptual explanations, architectural diagrams, and API breakdowns designed to be accessible to anyone with general programming knowledge.

🗺️ Documentation Directory Map

```
docs/
├── README.md                    # This Index
├── build.md                     # Build automation script (build.rs)
└── src/
    ├── main.md                  # Kernel entry point & bootstrap orchestrator (src/main.rs)
    ├── console.md               # Thread-safe console & formatting macros (src/console.rs)
    ├── arch/
    │   ├── mod.md               # Architecture-specific module root (src/arch/mod.rs)
    │   ├── gdt.md               # Global Descriptor Table, TSS & Rings (src/arch/gdt.rs)
    │   ├── cpu.md               # Control registers & CPU diagnostics (src/arch/cpu.rs)
    │   └── x86_64/
    │       └── boot.md          # Legacy BIOS boot assembly (header.asm, main.asm, main64.asm)
    ├── memory/
    │   ├── mod.md               # Memory management module root (src/memory/mod.rs)
    │   ├── frame_allocator.md   # Physical memory manager & 4 KiB frames (src/memory/frame_all
    │   └── paging.md            # Virtual memory & 4-level PML4 translation (src/memory/paging
    ├── capability/
    │   ├── mod.md               # Capability types & Rights bitflags (src/capability/mod.rs)
    │   └── table.md             # Task capability table & CSpace (src/capability/table.rs)
    └── ipc/
        ├── mod.md               # IPC subsystem root (src/ipc/mod.rs)
        ├── message.md           # Fast register payload & transferred caps (src/ipc/message.rs
        └── endpoint.md          # Message rendezvous channels (src/ipc/endpoint.rs)
```

📚 Module Reference Table

Subsystem	Source File	Documentation	Key Concepts & Analogies
Boot & Main	`src/main.rs`	docs/src/main.md	Entry point, boot sequence, hardware diagnostics, and panic handler.
Console I/O	`src/console.rs`	docs/src/console.md	Thread-safe `print!` / `println!` macros, UEFI screen color formatting.
GDT & TSS	`src/arch/gdt.rs`	docs/src/arch/gdt.md	Ring 0 / Ring 3 privilege levels, Task State Segment, double-fault stack.
CPU Control	`src/arch/cpu.rs`	docs/src/arch/cpu.md	Control registers (`CR0` , `CR2` , `CR3` , `CR4`), paging roots.
Arch Root	`src/arch/mod.rs`	docs/src/arch/mod.md	Architecture abstraction layer for hardware independence.
Physical Memory	`src/memory/frame_allocator.rs`	docs/src/memory/frame_allocator.md	UEFI memory map parsing, 4 KiB frame allocator ("parking lot attendant").
Virtual Memory	`src/memory/paging.rs`	docs/src/memory/paging.md	4-level x86_64 paging tree (PML4), address translation, page protection.
Memory Root	`src/memory/mod.rs`	docs/src/memory/mod.md	Physical and virtual memory coordinator.

Subsystem	Source File	Documentation	Key Concepts & Analogies
Capability Core	`src/capability/mod.rs`	docs/src/capability/mod.md	Security keycards (`Rights::READ` , `WRITE` , `EXECUTE` , `GRANT` , `CALL`).
Capability Table	`src/capability/table.rs`	docs/src/capability/table.md	Per-task CSpace keyring, permission checking, access revocation.
IPC Message	`src/ipc/message.rs`	docs/src/ipc/message.md	Registered courier package: 4 fast inline register arguments + security token.
IPC Endpoint	`src/ipc/endpoint.rs`	docs/src/ipc/endpoint.md	Postal mailbox rendezvous point, FIFO circular ring buffer.
IPC Root	`src/ipc/mod.rs`	docs/src/ipc/mod.md	Inter-process communication module export.
Build Automation	`build.rs`	docs/build.md	NASM assembly build script and link configuration.
Legacy Boot	`src/arch/x86_64/boot/`	docs/src/arch/x86_64/boot.md	Multiboot2 header and 32-bit to 64-bit assembly trampoline.

🚀 Boot Sequence Chronological Topic Order

When understanding or reviewing Hydra OS from initial hardware power-on through microkernel readiness, the documentation is structured sequentially:

1. **Firmware Handoff & Bootstrapping**: - Legacy Boot Trampoline & Long Mode Switch (Multiboot2 header, 32-bit to 64-bit Long Mode switch) - Build Automation & Assembly Pipeline (NASM toolchain and UEFI link settings)

2. **UEFI Entry & Bootstrap Orchestration**: - Kernel Entry Point & Bootstrap Pipeline (UEFI `main()`, panic handler, subsystem coordination, halt loop)
3. **Early Diagnostic Output**: - UEFI Console & Formatting (`print!`, `println!`, thread-safe `CONSOLE` spinlock, UEFI screen color formatting)
4. **CPU Architecture & Hardware Isolation (Step 1 & 2)**: - Architecture Abstraction Layer (Hardware abstraction root) - Global Descriptor Table (GDT) & TSS (Ring 0 / Ring 3 privilege levels, Task State Segment, Double Fault IST stack) - CPU Control Registers & Diagnostics (CR0 protection, CR2 fault addresses, CR3 PML4 paging root, CR4 extensions)
5. **Physical & Virtual Memory Management (Step 3 & 4)**: - Memory Subsystem Coordinator (Memory management root) - Physical Memory & Frame Allocator (UEFI memory map parsing, 1 MiB low memory reservation, 4 KiB frame allocator) - Virtual Memory & 4-Level Paging (x86_64 PML4 translation, 2 MiB / 1 GiB huge page decoding)
6. **Capability Security Model (Step 5)**: - Capability Security Primitives (Security keycards, `Rights` bitflags, `CapType` kernel resource identifiers) - Task Capability Space & Table (Per-task CSpace table, fixed 256-slot array storage, rights verification)
7. **Inter-Process Communication (Step 6)**: - IPC Subsystem Root (Microkernel message passing architecture) - IPC Fast Register Messages (4 fast inline register payload words, labels, transferred capability tokens) - IPC Endpoints & Mailbox Rendezvous (Circular FIFO ring buffers, rendezvous channels, thread-safe `SharedEndpoint`)

📄 Generating the Unified Documentation PDF

You can generate a single, professional PDF manual containing all documentation files, automatically formatted and ordered by the boot sequence topics:

Windows:

```
generate_pdf.bat
```

or

```
python scripts\generate_docs_pdf.py
```

Linux / WSL / macOS:

```
make docs-pdf
```

or

```
python3 scripts/generate_docs_pdf.py
```

The compiled PDF is generated at `docs/Hydra_OS_Documentation.pdf` and mirrored to `dist/Hydra_OS_Documentation.pdf`. An HTML version suitable for browser inspection or custom printing is also created at `docs/Hydra_OS_Documentation.html`.

BUILD & TOOLCHAIN • BOOT SEQUENCE ORDER #02

Build Script Documentation

- **Source File**: `build.rs`
- **Subsystem**: Toolchain & Assembly Build Automation

---

1. Purpose & Context

In earlier phases or legacy configurations, Hydra supported a hybrid BIOS/Multiboot bootloader mode that required compiling 16-bit and 32-bit assembly files (`header.asm` , `main.asm` , `main64.asm`) with the `nasm` assembler before linking into ELF objects.

In current native UEFI builds, Cargo compiles Rust directly targeting `x86_64-unknown-uefi` without requiring external assembly compilation, with `build = false` in `Cargo.toml` .

---

2. Key Functions

`assemble_file(file: &str, output_dir: &str)`

1. Checks the `DEBUG` environment variable to determine whether to emit DWARF debug symbols (`-g`).
2. Invokes `nasm` to compile an `.asm` source file into an ELF64 object (`.o`).
3. Instructs Cargo to re-run the build script if the source assembly file changes (`cargo:rerun-if-changed`).
4. Tells the Rust linker to link the resulting object file (`cargo:rustc-link-object`).

`assemble_all(output_dir: &str)`

Loops through the standard assembly boot files: - `boot/header.asm` : Multiboot2 header definition. - `boot/main.asm` : 32-bit protected mode entry point, page table setup, and long mode switch. - `boot/main64.asm` : 64-bit trampoline to Rust entry point.

STAGE 0: FIRMWARE HANDOFF • BOOT SEQUENCE ORDER #03

Legacy Assembly Bootstrapper Documentation

- **Source Directory**: `src/arch/x86_64/boot/`
- **Files**:
- `header.asm`
- `main.asm`
- `main64.asm`
- **Subsystem**: Architecture / Legacy BIOS Boot Trampoline

---

1. Overview

Before migrating to native **UEFI**, x86 PCs booted via legacy **BIOS**. Under BIOS: 1. The CPU boots in **16-bit Real Mode**. 2. A bootloader (like GRUB) loads the kernel into **32-bit Protected Mode**. 3. The kernel itself is responsible for setting up initial page tables, enabling PAE, switching to **64-bit Long Mode**, and jumping to 64-bit code.

These assembly files implement that transition for reference or hybrid BIOS/GRUB boot media.

---

2. File Breakdown

1. `header.asm` (Multiboot2 Header)

- Defines the magic numbers and architecture tags required by Multiboot2-compliant bootloaders (such as GRUB).
- Allows GRUB to recognize the kernel file as bootable.

2. `main.asm` (32-Bit to 64-Bit Transition)

- **Stack Initialization**: Allocates an initial boot stack.
- **CPU Checks**:
- `check_multiboot`: Verifies that the bootloader passed the expected Multiboot magic value in `EAX`.
- `check_cpuid`: Verifies that the CPU supports the `cpuid` instruction.
- `check_long_mode`: Queries CPUID to verify 64-bit Long Mode hardware support.
- **Identity Paging**: Configures an initial 2 MiB identity-mapped page table so memory addresses map 1:1 during the switch.
- **Enabling Long Mode**: Sets the Physical Address Extension bit (`PAE`) in `CR4`, enables Long Mode in the Extended Feature Enable Register (`EFER` MSR), and enables Paging in `CR0`.
- **GDT Setup**: Loads a temporary 64-bit code segment and performs a far jump to `long_mode_start`.

3. `main64.asm` (64-Bit Trampoline)

- Re-zeros data segment registers (`DS` , `ES` , `FS` , `GS` , `SS`).
- Invokes the Rust 64-bit entry point (`call kernel_main`).
- Halts the CPU if the function ever returns.

STAGE 1: KERNEL ENTRY POINT • BOOT SEQUENCE ORDER #04

Kernel Entry Point & Bootstrapping Documentation

- **Source File**: `src/main.rs`
- **Subsystem**: Microkernel Bootstrap & Initialization Orchestrator

---

1. Conceptual Overview

`src/main.rs` is the primary entry point executed when the UEFI firmware finishes hardware Power-On Self-Test (POST) and transfers control to `BOOTX64.EFI`.

Because Hydra targets native 64-bit UEFI directly: - The CPU is already in **64-bit Long Mode**. - Paging is enabled by the UEFI firmware. - The kernel receives a valid `uefi::prelude::SystemTable` reference, enabling it to access firmware services before taking full ownership of the machine.

---

2. Boot Execution Pipeline

```
graph TD
    A[UEFI Firmware POST] ⟶ B[main entry point]
    B ⟶ C[Initialize Console & Clear Screen]
    C ⟶ D[arch::gdt::init: Load GDT, TSS, Ring 0/3 Segments]
    D ⟶ E[arch::cpu::read_state: Inspect CR0, CR3 PML4 Root, CR4]
    E ⟶ F[memory::frame_allocator: Parse UEFI Memory Map & Test Frame]
    F ⟶ G[memory::paging: Test PML4 Virtual-to-Physical Translation]
    G ⟶ H[capability: Setup Task CSpace & Capability Table]
    H ⟶ I[ipc: Test Endpoint Message Rendezvous]
    I ⟶ J[Halt Loop with Interrupts Disabled]
```

---

3. Step-by-Step Code Walkthrough

The Panic Handler

```
#[panic_handler]
fn panic(_info: &PanicInfo) -> ! {
    console::set_color(Color::LightRed, Color::Black);
    println!("\n[!] KERNEL PANIC: {:?}", _info);
    x86_64::instructions::interrupts::disable();
    loop {
        x86_64::instructions::hlt();
    }
}
```

If an assertion fails or an unexpected unrecoverable condition occurs, the panic handler prints diagnostic error messages in red to the UEFI console, disables interrupts to prevent re-entrancy, and halts the CPU in a low-power

infinite loop (`hlt`).

Step 1: Segmentation & Hardware Privilege Setup

```
console::set_color(Color::Yellow, Color::Black);
println!("[*] Step 1: Initializing GDT, TSS & Ring 3 Segments...");
arch::gdt::init();
console::set_color(Color::LightGreen, Color::Black);
println!("[+] GDT loaded successfully.");
```

Disables interrupts and installs Hydra's custom Global Descriptor Table and Task State Segment, configuring the CPU for both supervisor (Ring 0) and isolated userland (Ring 3) execution.

Step 2: CPU Diagnostics

```
let cpu = arch::cpu::read_state();
println!("    - CR0: {:?}", cpu.cr0);
println!("    - CR3 (Active PML4 Root): {:#x}", cpu.cr3_frame.as_u64());
println!("    - CR4: {:?}", cpu.cr4);
```

Reads control registers to log the active hardware configuration, including the physical address of the initial root page table (`CR3`).

Step 3: Physical Memory Discovery & Frame Allocation

```
console::set_color(Color::Yellow, Color::Black);
println!("\n[*] Step 2: Discovering Memory & Initializing Frame Allocator...");
memory::frame_allocator::init_memory_from_uefi();

let (usable_mb, allocated_kb) = {
    let alloc = memory::frame_allocator::FRAME_ALLOCATOR.lock();
    (
        alloc.total_usable_memory() / (1024 * 1024),
        alloc.allocated_memory() / 1024,
    )
};
```

Queries UEFI firmware for the memory map, identifies all conventional RAM regions (skipping the first 1 MiB to protect BIOS/IVT data and null pointers), and initializes the kernel's physical frame allocator. It then tests allocating a sample 4 KiB physical frame.

Step 4: Virtual Memory Verification

```
console::set_color(Color::Yellow, Color::Black);
println!("\n[*] Step 3: Verifying 4-Level Paging (PML4) Address Translation...");
let address_space = memory::paging::AddressSpace::active();
let sample_virt = VirtAddr::new(main as *const () as usize as u64);
let sample_phys = unsafe { address_space.translate_addr(sample_virt) };
```

Walks the active 4-level PML4 page table hierarchy to verify that virtual addresses successfully resolve to physical memory frames, accommodating 4 KiB, 2 MiB, and 1 GiB page mappings.

Step 5: Capability Table Initialization

```
console::set_color(Color::Yellow, Color::Black);
println!("\n[*] Step 4: Initializing Task Capability Space...");
let mut cap_table = CapabilityTable::new();
let root_cap = Capability::new(
    CapType::AddressSpace {
        root_cr3: cpu.cr3_frame.as_u64(),
    },
    Rights::ALL,
);
let slot0 = cap_table.insert(root_cap).unwrap();
let ep_cap = Capability::new(
    CapType::Endpoint { endpoint_id: 1 },
    Rights(Rights::READ.0 | Rights::WRITE.0 | Rights::CALL.0),
);
let slot1 = cap_table.insert(ep_cap).unwrap();
```

Establishes the process capability table (CSpace), assigning granular access tokens to kernel resources (Root CR3 address space and IPC Endpoint #1).

Step 6: Microkernel IPC Rendezvous

```
console::set_color(Color::Yellow, Color::Black);
println!("\n[*] Step 5: Testing Microkernel IPC Rendezvous...");
let mut endpoint = Endpoint::new(1);
let msg = Message::with_args(0x1001, [42, 1337, 0, 0]).with_cap(ep_cap);
endpoint.send(msg).unwrap();
if let Some(received) = endpoint.receive() { ... }
```

Demonstrates inter-process communication: sending a message with inline arguments and an attached delegated capability, followed by receiving and consuming the message from the endpoint queue.

Step 7: Subsystems Operational & Terminal Halt Loop

```
console::set_color(Color::LightCyan, Color::Black);
println!("\n═══════════════════════════════════════════════════════════");
println!(" [+] HYDRA MICROKERNEL PHASE 1 SUBSYSTEMS OPERATIONAL!      ");
println!("═══════════════════════════════════════════════════════════");

x86_64::instructions::interrupts::disable();
loop {
    x86_64::instructions::hlt();
}
```

Upon successful boot and subsystem diagnostics, the kernel enters a halted state awaiting hardware interrupts or process scheduling.

STAGE 2: EARLY DIAGNOSTICS • BOOT SEQUENCE ORDER #05

Console & Diagnostic Output Documentation

- **Source File**: `src/console.rs`
- **Subsystem**: Core I/O & Formatting

---

1. Conceptual Overview

In a bare-metal `#![no_std]` environment, Rust's standard library (`std::io`, `std::println!`) does not exist because there is no underlying operating system yet.

The `console` module bridges Rust's `core::fmt` formatting system directly to UEFI's standard text output protocol (`SimpleTextOutput`), providing thread-safe `print!` and `println!` macros that can be used everywhere across the kernel.

---

2. Architecture & Concurrency

```
Any Kernel Thread / Subsystem
             |
             v
println!("Text: {}", value)
             |
             v
     CONSOLE (Mutex)  <--- Thread-safe synchronization via spin::Mutex
             |
             v
   uefi::system::with_stdout
             |
             v
   Physical Video Display / UEFI Console Screen
```

- `spin::Mutex`: Used instead of standard OS mutexes because interrupts or threads can acquire the lock without requiring OS scheduling services.

---

3. API Reference & Methods

`print!` and `println!` Macros

Drop-in replacements for standard Rust printing macros.

```
print!("Initializing subsystem ... ");
println!("Done: {} frames free.", free_count);
```

`console::set_color(foreground: Color, background: Color)`

Changes the active text color and background color on the UEFI screen.

```
use uefi::proto::console::text::Color;

console::set_color(Color::LightGreen, Color::Black);
println!("[+] Operation Succeeded");
```

`console::clear_screen()`

Clears the active screen buffer and resets the cursor to the top-left coordinate `(0, 0)`.

`console::CONSOLE` and `_print`

```
pub static CONSOLE: Mutex<Console> = Mutex::new(Console);

#[doc(hidden)]
pub fn _print(args: fmt::Arguments) {
    let mut console = CONSOLE.lock();
    let _ = console.write_fmt(args);
}
```

`CONSOLE` is a globally accessible, mutex-locked unit struct implementing `core::fmt::Write`. The `print!` and `println!` macros format arguments and invoke `_print`, acquiring the lock to safely serialize output to the UEFI `SimpleTextOutput` protocol.

STAGE 3: CPU ARCHITECTURE • BOOT SEQUENCE ORDER #06

Architecture Module Overview

- **Source File**: `src/arch/mod.rs`
- **Subsystem**: Architecture-Specific Primitives (x86_64)

---

1. Role in Hydra OS

The `arch` module isolates all processor-specific and machine-dependent code away from the rest of the generic microkernel.

When Hydra is ported to other CPU architectures (such as ARM64 or RISC-V), only the implementations inside `arch` need to be swapped out.

---

2. Submodules

- `gdt.rs`: Manages the Global Descriptor Table, Ring 0 / Ring 3 segmentation, and Task State Segment (TSS).
- `cpu.rs`: Provides interfaces for reading x86 control registers (`CR0` , `CR2` , `CR3` , `CR4`).

STAGE 3: CPU ARCHITECTURE • BOOT SEQUENCE ORDER #07

Global Descriptor Table (GDT) & TSS Documentation

- **Source File**: `src/arch/gdt.rs`
- **Subsystem**: Architecture / CPU Privilege & Segmentation

---

1. Conceptual Overview

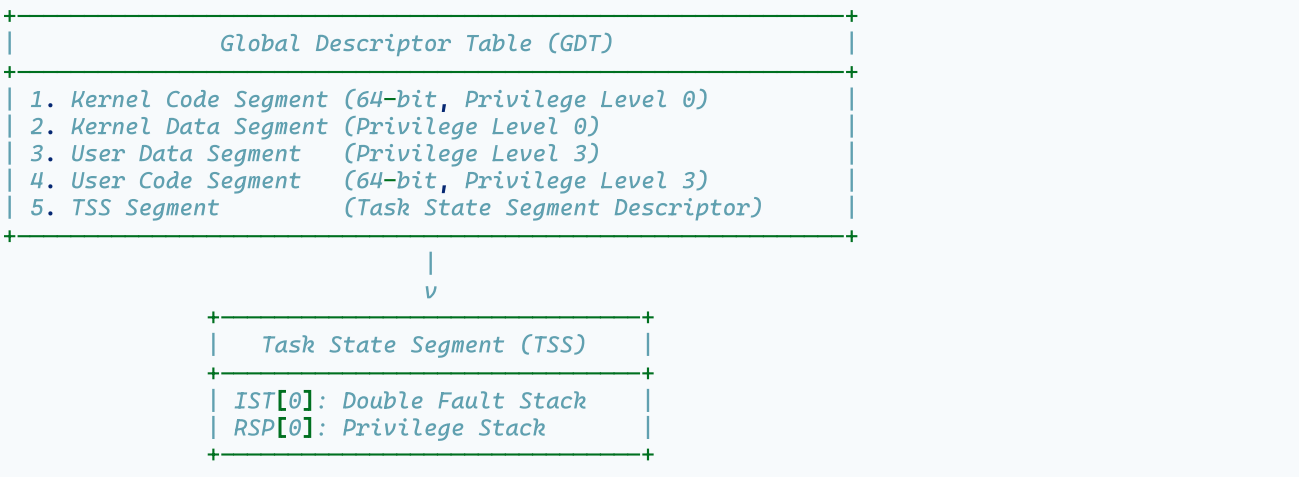
On x86_64 processors, the **Global Descriptor Table (GDT)** is a legacy hardware structure that remains vital for two critical operating system responsibilities: 1. **Privilege Levels (Rings)**: Defining segments for **Ring 0 (Kernel Mode)** and **Ring 3 (Userland Mode)**. 2. **Task State Segment (TSS)**: Defining emergency hardware stack pointers for CPU privilege transitions and catastrophic fault recovery (Double Faults).

Privilege Rings

- **Ring 0**: Full CPU privilege. The kernel executes here and can run privileged instructions (`cli` , `sti` , `invlpg` , write to control registers `CR0..CR4`).
- **Ring 3**: Sandboxed user mode. Applications run here and are restricted by the MMU and capability system.

---

2. Architecture & Components



---

3. Data Structures & Stacks

Stack Allocation

```
const STACK_SIZE: usize = 4096 * 5; // 20 KiB
static mut DOUBLE_FAULT_STACK: [u8; STACK_SIZE] = [0; STACK_SIZE];
static mut PRIVILEGE_STACK: [u8; STACK_SIZE] = [0; STACK_SIZE];
```

- `DOUBLE_FAULT_STACK`: When a double fault occurs (e.g. a page fault inside a kernel stack overflow), the CPU cannot use the current corrupted stack. Using the **Interrupt Stack Table (IST index 0)**, the CPU automatically switches to this clean stack.
- `PRIVILEGE_STACK`: When a Ring 3 user program triggers a system call or interrupt to enter Ring 0, the CPU loads this stack into `RSP` (`tss.privilege_stack_table[0]`).

`Selectors`

Stores the segment selectors returned when the segments are appended to the GDT:

```
pub struct Selectors {
    pub kernel_code_selector: SegmentSelector,
    pub kernel_data_selector: SegmentSelector,
    pub user_data_selector: SegmentSelector,
    pub user_code_selector: SegmentSelector,
    pub tss_selector: SegmentSelector,
}
```

---

4. API Reference & Methods

`arch::gdt::init()`

Initializes and activates the hardware segmentation structures: 1. **Disables interrupts** (`interrupts::disable()`) prior to swapping descriptor tables to ensure timer IRQs or external interrupts cannot trigger while segment registers hold transient or invalid selectors. 2. **Loads the GDT** into the CPU via `lgdt`. 3. **Reloads segment registers**: - `CS::set_reg(kernel_code_selector)` sets the code segment. - `DS`, `ES`, `SS` are reloaded with `kernel_data_selector`. - `FS` and `GS` are loaded with `SegmentSelector::NULL`. 4. **Loads the Task State Segment (TSS)** into the Task Register using `load_tss(tss_selector)`.

```
arch::gdt::init();
```

`arch::gdt::get_selectors() → &'static Selectors`

Provides access to the active segment selectors (`kernel_code_selector`, `kernel_data_selector`, `user_data_selector`, `user_code_selector`, `tss_selector`) for subsequent userland process spawning, context switches, and interrupt returns.

STAGE 3: CPU ARCHITECTURE • BOOT SEQUENCE ORDER #08

CPU Control & Register Diagnostics Documentation

- **Source File**: `src/arch/cpu.rs`
- **Subsystem**: Architecture / CPU Control Registers

---

1. Conceptual Overview

On x86_64 processors, the processor's operating mode, memory translation mechanisms, and security features are governed by specialized **Control Registers** (`CR0` , `CR2` , `CR3` , `CR4`):

- **CR0**: Controls basic execution features: Protected Mode, Paging enable, Write Protect (prevents kernel from inadvertently overwriting read-only user pages).
- **CR2**: Stores the linear virtual address that triggered the most recent **Page Fault** exception. Vital for demand paging and memory debugging.
- **CR3**: Points to the base physical address of the Level 4 Page Table (PML4). Loading a new value into CR3 immediately switches active virtual address spaces.
- **CR4**: Controls advanced architecture features: Physical Address Extension (PAE), Page Global Enable (PGE), OS support for SSE/AVX floating point, and User-Mode Instruction Prevention (UMIP).

---

2. The `CpuState` Struct

```rust
#[derive(Debug, Clone, Copy)]
pub struct CpuState {
    pub cr0: Cr0Flags,
    pub cr2: u64,
    pub cr3_frame: PhysAddr,
    pub cr4: Cr4Flags,
}
```

Captures an instantaneous snapshot of the machine's hardware configuration registers.

---

3. API Reference

`arch::cpu::read_state() → CpuState`

Queries the CPU hardware registers and packages them into a `CpuState` diagnostic struct.

```rust
use crate::arch::cpu;

let cpu = cpu::read_state();
println!("Paging root physical address: {:#x}", cpu.cr3_frame.as_u64());
```

STAGE 4: MEMORY SUBSYSTEM • BOOT SEQUENCE ORDER #09

Memory Subsystem Overview

- **Source File**: `src/memory/mod.rs`
- **Subsystem**: Memory Management

---

1. Role in Hydra OS

The `memory` module coordinates all physical and virtual memory resources required by the kernel and user processes.

It abstracts low-level hardware memory mechanisms (UEFI memory descriptors, x86_64 paging tables, MMU control registers) into safe, modular Rust interfaces.

---

2. Submodules

- `frame_allocator.rs`: Manages physical memory frames (4 KiB) discovered from the UEFI firmware memory map.
- `paging.rs`: Manages virtual memory address spaces and implements 4-level PML4 address translation.

STAGE 4: MEMORY SUBSYSTEM • BOOT SEQUENCE ORDER #10

Physical Memory & Frame Allocator Documentation

- **Source File**: `src/memory/frame_allocator.rs`
- **Subsystem**: Memory Management (Physical Layer)

---

1. Conceptual Overview

When a computer boots up, its physical Random Access Memory (RAM) is not one continuous empty space. Certain chunks are occupied by motherboard ROM, video graphics RAM, ACPI tables, and UEFI firmware code.

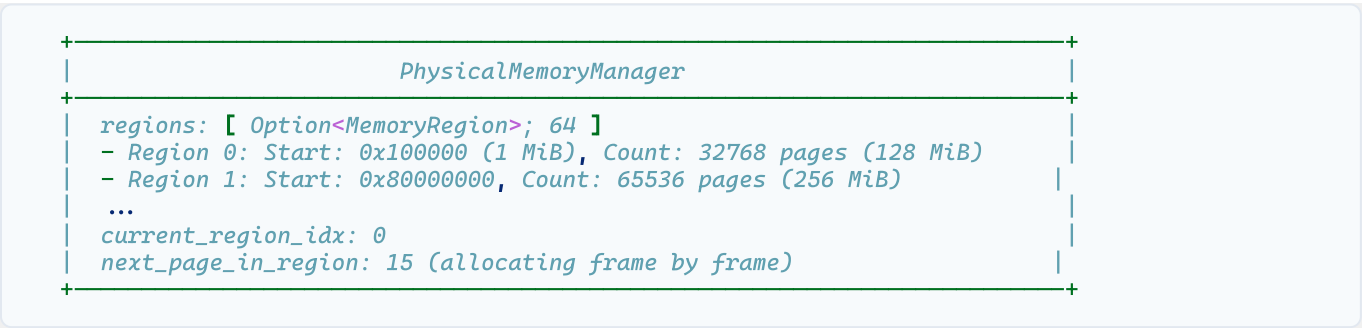
The **Physical Memory Frame Allocator** is responsible for: 1. Asking UEFI firmware which memory regions are safe and usable (`CONVENTIONAL` memory). 2. Dividing that usable RAM into standard **4096-byte (4 KiB) chunks**, called **Physical Frames**. 3. Handing out frames one-by-one to the kernel when it needs memory for page tables, stacks, or user processes.

The Real-World Analogy

Think of RAM as a vast parking lot: - Some spots are reserved for emergency vehicles or building infrastructure (firmware/hardware). - The **Physical Memory Manager** is the parking attendant with a map of all standard open spots. - Whenever a driver (a process or page table) arrives, the attendant assigns them the next available numbered spot (`0x1000` , `0x2000` , etc.).

---

2. Architecture & Data Structures



Key Constants

- `PAGE_SIZE: u64 = 4096` : Standard x86_64 base page size (4 KiB).
- `MAX_REGIONS: usize = 64` : Maximum number of non-contiguous usable RAM blocks tracked.

---

3. Struct Details

`MemoryRegion`

Represents a contiguous block of usable physical memory reported by firmware:

```
#[derive(Debug, Clone, Copy)]
pub struct MemoryRegion {
    pub start: PhysAddr,
    pub page_count: u64,
}
```

`PhysicalMemoryManager`

The state tracker for physical memory:

```
pub struct PhysicalMemoryManager {
    regions: [Option<MemoryRegion>; MAX_REGIONS],
    region_count: usize,
    current_region_idx: usize,
    next_page_in_region: u64,
    total_usable_bytes: u64,
    allocated_frames_count: u64,
}
```

---

4. API Reference & Methods

`PhysicalMemoryManager::add_region(&mut self, start: PhysAddr, page_count: u64)`

Registers a contiguous block of usable physical memory into the tracker.

`PhysicalMemoryManager::allocate_frame(&mut self) -> Option<PhysFrame<Size4KiB>>`

Finds the next free 4 KiB frame from the current memory region and advances the bump pointer. If a region is exhausted, it moves to the next region. - Returns `Some(PhysFrame)` containing the physical start address. - Returns `None` if physical memory is exhausted (Out of Memory).

`PhysicalMemoryManager::total_usable_memory(&self) -> u64`

Returns total detected usable physical RAM in bytes.

`PhysicalMemoryManager::allocated_memory(&self) -> u64`

Returns total allocated physical RAM in bytes.

`PhysicalMemoryManager::free_memory(&self) -> u64`

Returns currently unallocated physical RAM in bytes.

`x86_64::structures::paging::FrameAllocator<Size4KiB>` Trait Implementation

```
unsafe impl FrameAllocator<Size4KiB> for PhysicalMemoryManager {
    fn allocate_frame(&mut self) -> Option<PhysFrame<Size4KiB>> {
        self.allocate_frame()
    }
}
```

Enables `PhysicalMemoryManager` to serve directly as the frame supplier for `x86_64` page table manipulators and memory allocators.

---

5. UEFI Firmware Integration & Low Memory Protection

```
pub fn init_memory_from_uefi()
```

Queries `uefi::boot::memory_map(MemoryType::LOADER_DATA)`: 1. Iterates through all memory descriptors provided by the UEFI firmware. 2. Filters specifically for `uefi::boot::MemoryType::CONVENTIONAL`. 3. **Reserves the First 1 MiB (`0x100000`)**: Skips addresses below 1 MiB to protect legacy Real-Mode Interrupt Vector Tables (IVT), BIOS Data Areas (BDA), and to prevent `PhysAddr(0)` from being issued as a valid frame (protecting against null-pointer aliasing). 4. Registers each usable conventional memory block into the global `FRAME_ALLOCATOR` mutex.

---

6. Practical Example

```
use crate::memory::frame_allocator::{FRAME_ALLOCATOR, init_memory_from_uefi};

// 1. Read UEFI firmware memory map and discover usable RAM
init_memory_from_uefi();

// 2. Allocate a 4 KiB physical frame
let frame = {
    let mut alloc = FRAME_ALLOCATOR.lock();
    alloc.allocate_frame().expect("System has available RAM")
};

// 3. Inspect frame address
let physical_address = frame.start_address().as_u64();
```

STAGE 5: VIRTUAL MEMORY • BOOT SEQUENCE ORDER #11

Virtual Memory & Paging Documentation

- **Source File**: `src/memory/paging.rs`
- **Subsystem**: Memory Management (Virtual Layer & Hardware MMU)

---

1. Conceptual Overview

Modern 64-bit CPUs do not let user programs or even high-level kernel functions read and write directly to physical RAM chips. Instead, they use **Virtual Memory**.

Every address a program interacts with is a **Virtual Address**. The hardware Memory Management Unit (MMU) consults a multi-level tree of translation tables (called **Page Tables**) to convert that virtual address into a **Physical Address** in RAM.

Benefits of Paging

1. **Isolation**: Process A and Process B can both have code at virtual address `0x400000`, but they map to completely different physical RAM chips.
2. **Protection**: Pages can be marked read-only, non-executable (NX/XD bit to prevent malware buffer-overflow attacks), or kernel-only.
3. **Continuity**: Large arrays can appear contiguous in virtual memory even if their physical frames are scattered across RAM.

---

2. 4-Level Paging Architecture (x86_64)

In 64-bit x86 architecture, translating a 48-bit canonical virtual address uses a 4-level tree:

```
Virtual Address:
[ PML4 Index (9 bits) | PDPT Index (9 bits) | PD Index (9 bits) | PT Index (9 bits) | Offset (12 b
        |                      |                    |                  |                    |
        v                      v                    v                  v                    v
   [ CR3 Register ]
        |
        +---> PML4 Table (Level 4)
                  |
                  +---> PDPT Table (Level 3 - Page Directory Pointer Table)
                            |
                            +---> PD Table (Level 2 - Page Directory)
                                      |
                                      +---> PT Table (Level 1 - Page Table)
                                                |
                                                +---> Physical Frame (4 KiB) + Offset
```

- **CR3**: The CPU control register holding the physical address of the active Level 4 (PML4) table.

3. The `AddressSpace` Struct

```
pub struct AddressSpace {
    pml4_phys: PhysAddr,
}
```

Represents a virtual memory context anchored by its PML4 root table.

---

4. API Reference & Methods

`AddressSpace::active() → Self`

Reads the CPU's `CR3` register to attach to the currently running address space.

```
let active_space = AddressSpace::active();
```

`AddressSpace::pml4_physical_address(&self) → PhysAddr`

Returns the physical address of the Level 4 Page Table.

`AddressSpace::table_at<'a>(&self, phys: PhysAddr) → &'a PageTable`

(Unsafe) Dereferences a physical address as an active `PageTable` reference. In native 64-bit UEFI environments, memory is identity-mapped, allowing direct address dereferencing during bootstrap.

`AddressSpace::translate_addr(&self, addr: VirtAddr) → Option<PhysAddr>`

Walks the 4-level page table hierarchy for a given virtual address: 1. Checks PML4 entry for `PRESENT`. 2. Checks PDPT entry for `PRESENT` and handles 1 GiB huge pages. 3. Checks PD entry for `PRESENT` and handles 2 MiB huge pages. 4. Checks PT entry for `PRESENT`. 5. Extracts the physical frame base and adds the 12-bit offset (`0..4095`). - Returns `Some(PhysAddr)` if the address is currently mapped. - Returns `None` if any table level in the walk is unmapped.

---

5. Practical Example

```
use crate::memory::paging::AddressSpace;
use x86_64::VirtAddr;

let space = AddressSpace::active();
let virt_ptr = VirtAddr::new(0x1000_5000);

// Translate to physical address:
match unsafe { space.translate_addr(virt_ptr) } {
    Some(phys) => println!("Virtual {:?} maps to Physical {:?}", virt_ptr, phys),
    None => println!("Virtual {:?} is not mapped in this address space", virt_ptr),
}
```

STAGE 6: SECURITY & CAPABILITIES • BOOT SEQUENCE ORDER #12

Capability Subsystem Overview

- **Source File**: `src/capability/mod.rs`
- **Subsystem**: Capability-Based Security

---

1. Conceptual Overview

Traditional operating systems (like standard Linux or Windows) often rely on **Access Control Lists (ACLs)** or ambient authority (e.g., "this process runs as root/administrator, so it can touch anything"). If an administrative process has a bug or is hijacked, the entire system is compromised.

Hydra uses Capability-Based Security (similar to seL4 and Fuchsia Zircon).

The Real-World Analogy

Think of a **Capability** as a non-forgeable, digital smart keycard: - You cannot just claim "I am the administrator". - You can only perform an operation if you hold a specific keycard that explicitly grants the right to perform that action on that exact resource. - You can hand a copy of a keycard with restricted permissions to another process (e.g., giving someone a key that allows reading a file, but not modifying or deleting it).

---

2. The `Rights` Bitflags

`Rights` defines what actions a process holding a capability is permitted to execute:

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct Rights(pub u32);
```

Supported Rights Flags

Flag	Value	Description
`Rights::READ`	`1 << 0` (1)	Permission to inspect data in the target object.
`Rights::WRITE`	`1 << 1` (2)	Permission to modify or overwrite the target object.
`Rights::EXECUTE`	`1 << 2` (4)	Permission to execute machine instructions from the object (memory frame).
`Rights::GRANT`	`1 << 3` (8)	Permission to delegate, duplicate, or transfer this capability to another process.
`Rights::CALL`	`1 << 4` (16)	Permission to invoke or call an IPC Endpoint.
`Rights::ALL`	`0x1F` (31)	Full combined permissions (`READ | WRITE | EXECUTE | GRANT | CALL`).

Checking Permissions

- `rights.contains(required)` : Returns `true` if all bits in `required` are present.

---

3. The `CapType` Enum

`CapType` defines what kind of physical or virtual kernel resource the capability represents:

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum CapType {
    UntypedMemory { size_bytes: usize },
    MemoryFrame { physical_addr: u64 },
    AddressSpace { root_cr3: u64 },
    Endpoint { endpoint_id: u32 },
    Thread { thread_id: u32 },
    HardwareDevice { device_id: u32 },
}
```

- `UntypedMemory` : Raw, unallocated physical memory blocks that can be carved up into specific objects.
- `MemoryFrame` : A specific 4096-byte page of physical RAM.
- `AddressSpace` : A virtual memory context (represented by the x86_64 CR3 root page table).
- `Endpoint` : A communication mailbox for IPC.
- `Thread` : A schedulable execution thread.
- `HardwareDevice` : An I/O hardware device (such as a GPU, network card, or timer).

---

4. The `Capability` Struct

```rust
#[derive(Debug, Clone, Copy)]
pub struct Capability {
    pub cap_type: CapType,
    pub rights: Rights,
}
```

Combines the resource descriptor (`cap_type`) with the allowable permissions (`rights`).

Constructor

```rust
let memory_cap = Capability::new(
    CapType::MemoryFrame { physical_addr: 0x1000 },
    Rights(Rights::READ.0 | Rights::WRITE.0)
);
```

STAGE 6: SECURITY & CAPABILITIES • BOOT SEQUENCE ORDER #13

Capability Table Module Documentation

- **Source File**: `src/capability/table.rs`
- **Subsystem**: Capability-Based Security

---

1. Conceptual Overview

In Hydra, userland processes cannot store raw pointers to kernel objects or hardware devices. Instead, the kernel maintains a **Capability Table** (also known as a **CSpace** or **CNode**) on behalf of each process.

The Real-World Analogy

Think of a **Capability Table** like a keyring or safe-deposit locker: - The process holds numbers (slots: `0`, `1`, `2`, ...). - When a process asks the kernel, "*Read from memory at Slot 3*", the kernel checks Slot 3 in that process's Capability Table. - If Slot 3 contains a valid capability and includes `Rights::READ`, the kernel performs the operation. - If the slot is empty or lacks the required right, the kernel immediately denies the request. - The process never touches the raw physical address or hardware pointer directly, preventing spoofing and unauthorized access.

---

2. Architecture & Constants



- `MAX_CAPABILITIES: usize = 256`: Each table can hold up to 256 active capabilities per task in fixed array storage, avoiding dynamic heap allocation.

---

3. The `CapabilityTable` Struct

```rust
pub struct CapabilityTable {
    slots: [Option<Capability>; MAX_CAPABILITIES],
    next_free_slot: usize,
}
```

- `slots`: Fixed array of 256 optional capability slots.
- `next_free_slot`: Circular search cursor to quickly find available slots upon insertion.

---

4. API Reference & Methods

`CapabilityTable::new() -> Self`

Creates a new, empty capability table where all 256 slots are `None`.

`CapabilityTable::insert(&mut self, cap: Capability) -> Result<usize, &'static str>`

Allocates the first available empty slot, stores the capability, and returns the slot index (`0..255`). - **Returns** `Ok(slot_index)` on success. - **Returns** `Err("Capability table full")` if all 256 slots are currently occupied.

`CapabilityTable::get(&self, slot: usize) -> Option<&Capability>`

Retrieves a read-only reference to the capability stored at the specified slot. - Returns `None` if the slot is out of bounds or empty.

`CapabilityTable::revoke(&mut self, slot: usize) -> Option<Capability>`

Removes and clears the capability at `slot`, effectively revoking the process's access to that resource.

`CapabilityTable::check_rights(&self, slot: usize, required_rights: Rights) -> bool`

Helper method that verifies: 1. Whether `slot` contains an active capability. 2. Whether the capability possesses all bits defined in `required_rights`.

---

5. Practical Example

```rust
use crate::capability::table::CapabilityTable;
use crate::capability::{Capability, CapType, Rights};

let mut cspace = CapabilityTable::new();

// 1. Grant access to a hardware memory frame
let frame_cap = Capability::new(
    CapType::MemoryFrame { physical_addr: 0x2000_0000 },
    Rights(Rights::READ.0 | Rights::WRITE.0)
);

// 2. Insert into the process table
let slot = cspace.insert(frame_cap).expect("Table has free slots");

// 3. Verify permissions before performing an operation
if cspace.check_rights(slot, Rights::WRITE) {
    // Permission granted: perform write
} else {
    // Permission denied: trigger error
}
```

STAGE 7: IPC SUBSYSTEM • BOOT SEQUENCE ORDER #14

IPC Subsystem Overview

- **Source File**: `src/ipc/mod.rs`
- **Subsystem**: Microkernel IPC (Inter-Process Communication)

---

1. Role in Hydra OS

The `ipc` module binds together the fundamental primitives for Inter-Process Communication in the Hydra microkernel.

Because Hydra isolates all system services (network stack, file systems, device drivers, cloud node daemons) into separate userland processes, fast and secure message passing is the backbone of the entire operating system.

---

2. Module Organization

- `endpoint.rs`: Provides the `Endpoint` and `SharedEndpoint` message queues.
- `message.rs`: Defines the structure of an IPC `Message`, including inline arguments and security capability tokens.

---

3. Re-exported Types

```
pub use endpoint::Endpoint;
pub use message::Message;
```

These re-exports allow any other module in the kernel to conveniently import IPC types directly via `use crate::ipc::{Endpoint, Message};`.

STAGE 7: IPC SUBSYSTEM • BOOT SEQUENCE ORDER #15

IPC Message Module Documentation

- **Source File**: `src/ipc/message.rs`
- **Subsystem**: Microkernel IPC (Inter-Process Communication)

---

1. Conceptual Overview

In a microkernel operating system like **Hydra**, core operating system services (such as file systems, network drivers, and device managers) run as separate, isolated programs rather than inside a single monolithic kernel. Because these isolated programs cannot directly access each other's memory, they must communicate by sending **Messages** across a secured boundary.

The Real-World Analogy

Think of an **IPC Message** like a registered letter or courier package: 1. **The Label (`label`)**: The subject line or action request (e.g., "Read File", "Get Time", "Allocate Memory"). 2. **The Fast Payload (`args`)**: A quick handwritten note with up to 4 numbers or pointers that fit directly into the CPU's registers for high-speed delivery without touching RAM. 3. **The Enclosed Key/Passport (`capability`)**: An optional security token granted to the receiver, giving them special permission to access a specific memory buffer, device, or channel.

---

2. Architecture & Data Structures

```
+-------------------------------------------------------------+
|                         Message                             |
+-------------------------------------------------------------+
| label: u32                | Protocol / Syscall ID           |
| args: [u64; 4]            | Up to 4 fast 64-bit parameters  |
| capability: Option<Cap>   | Optional transferred security   |
|                           | token / permission              |
+-------------------------------------------------------------+
```

Constants

- `MAX_INLINE_WORDS: usize = 4`: The number of 64-bit arguments (`u64`) carried directly within a message. Using 4 words (32 bytes) allows the microkernel to transfer arguments directly using CPU registers (such as `RDI`, `RSI`, `RDX`, `RCX` on x86_64), making IPC fast.

---

3. The `Message` Struct

```rust
#[derive(Debug, Clone, Copy)]
pub struct Message {
    pub label: u32,
    pub args: [u64; MAX_INLINE_WORDS],
    pub capability: Option<Capability>,
}
```

Fields Explained

Field	Type	Description
`label`	`u32`	An operation code indicating what action is requested (e.g. `0x1001` for ping, `0x2001` for read).
`args`	`[u64; 4]`	Array of 4 inline 64-bit numbers. Can be integers, memory offsets, or byte lengths.
`capability`	`Option<Capability>`	An optional security token transferred along with the message to delegate authority.

---

4. API Reference & Methods

`Message::new(label: u32) → Self`

Creates a new message with the given operation label. All 4 arguments are initialized to `0`, and no capability is attached.

```rust
let msg = Message::new(0x01); // Simple ping message
```

`Message::with_args(label: u32, args: [u64; MAX_INLINE_WORDS]) → Self`

Creates a new message with an operation label and up to 4 inline arguments.

```rust
// Example: requesting to read 512 bytes starting at offset 1024
let msg = Message::with_args(0x1002, [1024, 512, 0, 0]);
```

`Message::with_cap(mut self, cap: Capability) → Self`

Builder pattern method to attach a capability to an existing message. This transfers access rights to the receiving process.

```rust
let msg = Message::new(0x2000).with_cap(shared_memory_cap);
```

---

5. Practical Example

```rust
use crate::capability::{Capability, CapType, Rights};
use crate::ipc::Message;

// 1. Create a capability granting READ access to an endpoint
let ep_cap = Capability::new(CapType::Endpoint { endpoint_id: 10 }, Rights::READ);

// 2. Prepare an IPC message carrying data and delegating the capability
let message = Message::with_args(0x1001, [42, 1337, 0, 0])
    .with_cap(ep_cap);

// 3. The message is now ready to be sent over an Endpoint!
```

STAGE 7: IPC SUBSYSTEM • BOOT SEQUENCE ORDER #16

IPC Endpoint Module Documentation

- **Source File**: `src/ipc/endpoint.rs`
- **Subsystem**: Microkernel IPC (Inter-Process Communication)

---

1. Conceptual Overview

In a microkernel architecture, programs cannot talk directly to each other because their address spaces are completely isolated by the CPU's memory management unit (MMU). An **Endpoint** acts as a meeting place (rendezvous point) or postal mailbox managed by the kernel.

The Real-World Analogy

Imagine an apartment building where residents cannot enter each other's apartments: - An **Endpoint** is a secure PO Box in the lobby. - Process A drops a `Message` into the box (`send`). - Process B checks the box and retrieves the message (`receive`). - The kernel ensures only residents holding the correct security key (`Capability`) can post or retrieve items from that specific box.

---

2. Architecture & Data Structures



Queue Parameters

- `QUEUE_CAPACITY: usize = 16` : The maximum number of unread messages the endpoint buffer can hold before rejecting new incoming messages with an error.

---

3. Struct Details

`Endpoint`

The core internal endpoint managing a circular ring buffer of messages.

```
pub struct Endpoint {
    id: u32,
    buffer: [Option<Message>; QUEUE_CAPACITY],
    head: usize,
    tail: usize,
    count: usize,
}
```

- `id`: A unique numerical identifier for this endpoint.
- `buffer`: Fixed-size array holding pending messages without needing dynamic heap allocations.
- `head`: Read pointer pointing to the next message to be consumed (FIFO order).
- `tail`: Write pointer pointing to the next free slot for an incoming message.
- `count`: Current number of queued, unread messages.

`EndpointState`

Indicates the scheduling and readiness state of the communication channel:

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum EndpointState {
    Idle,
    Active,
}
```

`SharedEndpoint`

A thread-safe wrapper around `Endpoint` using a `spin::Mutex`. This allows concurrent threads and interrupts to safely send and receive messages without race conditions.

```
pub struct SharedEndpoint(Mutex<Endpoint>);
```

---

4. API Reference & Methods

`Endpoint::new(id: u32) -> Self`

Initializes a new empty message queue with the specified ID.

`Endpoint::send(&mut self, msg: Message) -> Result<(), &'static str>`

Pushes a message into the queue: - **Returns** `Ok(())` if the message was placed into the buffer. - **Returns** `Err("Endpoint message queue is full")` if all 16 slots are occupied.

`Endpoint::receive(&mut self) -> Option<Message>`

Pulls the oldest message out of the queue (FIFO): - **Returns** `Some(Message)` if a message is available. - **Returns** `None` if the queue is currently empty.

`Endpoint::pending_count(&self) -> usize`

Returns how many messages are waiting to be read.

`SharedEndpoint::new(id: u32) → Self`

Constructs a new mutex-protected endpoint.

`SharedEndpoint::send(&self, msg: Message) → Result<(), &'static str>`

Acquires the spinlock and sends a message into the underlying endpoint.

`SharedEndpoint::receive(&self) → Option<Message>`

Acquires the spinlock and consumes the next available message.

`SharedEndpoint::id(&self) → u32`

Returns the ID of the wrapped endpoint.

---

5. Practical Example

```
use crate::ipc::{Endpoint, Message};

// 1. Create a communication endpoint with ID 1
let mut mailbox = Endpoint::new(1);

// 2. Sender prepares and sends a message
let request = Message::with_args(0x100, [1, 2, 3, 4]);
mailbox.send(request).expect("Queue has space");

// 3. Check queue status
assert_eq!(mailbox.pending_count(), 1);

// 4. Receiver reads the message
if let Some(msg) = mailbox.receive() {
    // Process msg.label and msg.args ...
}
```